

Key Algorithms

Algorithm 1 BuildFlowNodes

```
1: function BUILDFLOWNODES(pages, visitedSet, unlockedSet, endingsSet)
2:   visible  $\leftarrow$  visitedSet  $\cup$  unlockedSet
3:   for all  $p \in$  new copy of visible do
4:     if  $p.choice1 \neq \text{null}$  then visible.add( $p.choice1$ )
5:     if  $p.choice2 \neq \text{null}$  then visible.add( $p.choice2$ )
6:   nodeMap  $\leftarrow$  empty map; nodes  $\leftarrow$  empty list
7:   for all  $p$  in pages where  $p.id \in$  visible do
8:     isVisited  $\leftarrow p.id \in$  visitedSet
9:     isEnding  $\leftarrow$  "Ending :" +  $p.id \in$  endingsSet
10:    isDeath  $\leftarrow p.isDeath$ 
11:    node  $\leftarrow$  FlowNode( $p.id, 0, 0, isVisited, isEnding$ ); node.death  $\leftarrow$  is-
      Death
12:    node.children  $\leftarrow$  empty list
13:    nodeMap[ $p.id$ ]  $\leftarrow$  node; nodes.append(node)
14:    for all node in nodes do
15:      for  $c$  in { $node.id.choice1, node.id.choice2$ } do
16:        if  $c \neq \wedge c \in$  nodeMap then
17:          node.children.append( $c$ )
18:  return nodes
```

Algorithm 2 AssignCoordinates

```
1: function ASSIGNCOORDINATES(nodeMap, nodeId, depth, offset, visited)
2:   if nodeId ∈ visited then return offset
3:   visited.add(nodeId)
4:   node ← nodeMap[nodeId];
5:   if node is null then return offset
6:    $x \leftarrow \text{depth} \times X\_SPACING$ 
7:    $y \leftarrow \text{offset} \times Y\_SPACING$ 
8:   if node.isDeath then  $y \leftarrow y - \frac{Y\_SPACING}{2}$ 
9:   node.x ←  $x$ ; node.y ←  $y$ 
10:  if node.children is empty then return offset+1
11:  children.sort()
12:  mainChild ← children[0]
13:  ASSIGNCOORDINATES(nodeMap, mainChild, depth+1, offset, visited)
14:  branchOffset ← offset+1
15:  for  $i = 1$  to children.size − 1 do
16:    child ← children[i]
17:    branchOffset ← ASSIGNCOORDINATES(nodeMap, child, depth+1,
    branchOffset, visited)
18:  return branchOffset
```

Complexity Analysis

Let $n = |\text{visible}|$ and $m = \sum |\text{children}|$. In a tree $m \leq n - 1$.

$$T_{\text{build}}(n, m) = \underbrace{O(n)}_{\text{collect visible}} + \underbrace{O(n)}_{\text{create nodes}} + \underbrace{O(m)}_{\text{wire edges}} = O(n + m) = O(n)$$

$$T_{\text{assign}}(n, m) = \sum_{v \in V} (O(1) + O(\deg(v))) = O\left(\sum_v 1 + \sum_v \deg(v)\right) = O(n + m) = O(n)$$

Proof Sketch by Induction

Base case ($n = 1$): A single node—both algorithms do $O(1)$ work.

Inductive step: Assume for all $k < n$ the routines run in $O(k)$. Adding one more node:

- BUILDFLOWNODES performs a constant amount of extra work to include the new node and its two potential child links.

- ASSIGNCOORDINATES visits the new node exactly once, does $O(1)$ local work plus recursive calls to its (at most two) children, each maintaining the inductive bound.

Thus by induction both are $O(n)$.